

The MVC Segmentation Pipeline: A Complete Beginner's Guide

Part 1: Understanding the Problem

What is EMG and MVC?

EMG (Electromyography) is a technique that records the electrical activity produced by your muscles. When you contract a muscle (like flexing your bicep), motor neurons fire electrical signals to tell the muscle fibers to contract. These tiny electrical signals can be picked up by sensors placed on the skin.

Think of it like this: imagine your muscle is a stadium full of people. When everyone is quiet (muscle at rest), there's just a low background murmur. When they all start cheering (muscle contracting), you get a loud roar. The EMG sensor is like a microphone recording that sound.

MVC (Maximum Voluntary Contraction) is when you contract a muscle as hard as you possibly can. In research studies, we ask participants to do this so we can measure their "100% effort" level. Later, when they do normal activities, we can express their muscle effort as a percentage of their MVC (e.g., "during typing, the forearm muscle worked at 15% of maximum").

The Protocol Problem

In our study, participants perform **two MVC contractions** during a calibration session:

1. They rest for a few seconds
2. They squeeze/contract as hard as they can for 2-3 seconds
3. They rest again
4. They squeeze again for 2-3 seconds
5. They rest

The recording might be 30-75 seconds long, and somewhere in there are these two important "peaks" of maximum effort. **Our job is to automatically find where those two peaks are.**

Why is This Hard?

Several challenges make automatic detection tricky:

1. **Noise:** EMG signals are inherently noisy. Even at "rest," there's always some electrical activity (from other muscles, from the electronics, from movement).
2. **Variable timing:** Participants don't always follow instructions perfectly. The MVCs might happen at different times in different recordings.
3. **Signal artifacts:** Sometimes there are spurious signals from electrode movement, cable tugging, or other interference.
4. **Different signal levels:** One person's "maximum effort" signal amplitude might be very different from another person's.

Part 2: The Raw Signal

What Does Raw EMG Look Like?

If you looked at a raw EMG signal, you'd see something like a wiggly line that looks almost like random noise. Here's why:

- **At rest:** Small random fluctuations around zero (like static on an old TV)
- **During contraction:** Larger, more intense fluctuations (like aggressive static)

The signal isn't a smooth wave like a heartbeat—it's chaotic because it's the sum of many motor units firing at different times. The key insight is that **during stronger contractions, the fluctuations are bigger (higher amplitude)**.

The Signal in Numbers

Our EMG is recorded in **millivolts (mV)**—thousandths of a volt. Typical values might be:

- Rest: fluctuations of ± 0.05 mV
- MVC: fluctuations of ± 0.5 to ± 2 mV

The signal is sampled at **1000 Hz** (1000 measurements per second), so a 60-second recording has 60,000 data points.

Part 3: Step-by-Step Pipeline

Now let's walk through each processing step in detail.

STEP 1: Preprocessing (Cleaning the Signal)

1A: DC Offset Removal

What it is: The raw signal might not be centered around zero. It might fluctuate around 0.1 mV or -0.05 mV due to electrode chemistry or amplifier drift.

What we do: Subtract the average value from every point.

Analogy: Imagine you're measuring heights of students, but your ruler starts at 5cm instead of 0. Every measurement is 5cm too high. Subtracting 5 from each measurement "re-centers" your data.

Math:

```
signal_centered = signal - mean(signal)
```

If the signal averaged 0.02 mV, every point now has 0.02 subtracted, centering the signal around zero.

1B: Bandpass Filtering (20-500 Hz)

What it is: We keep only frequencies between 20 and 500 Hz, removing everything else.

Why 20 Hz as the lower limit?

- Below 20 Hz, you get **motion artifacts**—slow wobbles from the electrode moving on the skin, from breathing, from the arm swaying. These aren't muscle signals.
- The actual muscle electrical activity has frequencies above 20 Hz.

Why 500 Hz as the upper limit?

- Above 500 Hz, there's mostly **electronic noise** from the amplifier, power lines (50/60 Hz harmonics), and other interference.
- Real muscle signals don't contain much information above 500 Hz.

Analogy: Imagine you're at a concert trying to hear the singer (the muscle signal), but there's also low rumbling from traffic outside (motion artifacts) and high-pitched electronic feedback (noise). A bandpass filter is like noise-canceling headphones that let through only the frequencies where the singing lives.

Technical detail: We use a **Butterworth filter** of order 4. This is a mathematical design that smoothly attenuates unwanted frequencies without creating weird distortions. The "order" affects how sharply it cuts off—higher order = sharper cutoff.

STEP 2: Energy Extraction (TKEO)

Now we have a clean(er) EMG signal, but it's still a wiggly line that goes positive and negative. We need to convert this into a measure of "how much activity is happening at each moment."

2A: What is TKEO?

TKEO stands for **Teager-Kaiser Energy Operator**. It's a mathematical formula that estimates the "instantaneous energy" of a signal.

The formula:

$$\psi[n] = x[n]^2 - x[n-1] \cdot x[n+1]$$

In plain English: For each point in the signal, take its value squared, then subtract the product of its neighbors.

Why this works:

The TKEO is sensitive to both **amplitude** (how big the signal is) and **frequency** (how fast it's wiggling). During muscle contraction:

- The amplitude increases (bigger fluctuations)
- The frequency content changes (more high-frequency activity)

Both of these make TKEO output larger values.

Intuition: Imagine you're watching a rope being shaken.

- If someone shakes it gently and slowly → low energy
- If someone shakes it hard and fast → high energy

TKEO captures this "shaking intensity."

Why TKEO instead of just squaring the signal?

Simply squaring the signal (to make everything positive) also works, but TKEO has a key advantage: it produces **sharper transitions** at the start and end of contractions. This makes it easier to detect exactly when the MVC begins and ends.

Think of it like this: a regular squared signal is like a photograph with a bit of blur. TKEO is like running a "sharpening" filter that makes edges crisper.

2B: Rectification

TKEO can produce negative values in some cases (when the signal is nearly constant). We **rectify** by setting any negative values to zero.

```
energy = max(0, TKEO_output)
```

This ensures our energy measure is always positive or zero.

2C: Windowed Sum (50ms Windows)

The TKEO output is still quite "spiky"—it jumps around from sample to sample. To get a smoother estimate of energy, we compute the **sum of TKEO values over a sliding window**.

Window size: 50 milliseconds = 50 samples (at 1000 Hz)

How it works:

1. Take samples 1-50, add them up → energy at time point 25
2. Take samples 2-51, add them up → energy at time point 26
3. Continue sliding the window across the signal...

Why sum and not average?

- **Sum** gives us "total energy in this 50ms window"
- **Average** would give us "average energy per sample"

Both work, but sum preserves the intuitive interpretation: "how much total electrical activity happened in this brief moment."

Analogy: Imagine counting cars passing a checkpoint. You could report:

- "457 cars passed in the last minute" (sum)
- "7.6 cars per second" (average)

Both are valid, but the sum is more intuitive for "how busy was that minute."

STEP 3: Log Transform

At this point, we have an "energy" signal that's high during MVCs and low during rest. But there's a problem: the range of values is enormous.

The problem:

- Rest energy might be: 0.001
- MVC energy might be: 100

That's a difference of **100,000x**! This huge range makes it hard to set a threshold that works for both detecting MVCs and ignoring noise.

The solution: Take the **logarithm** (base 10).

$$\log_energy = \log_{10}(energy + \epsilon)$$

(We add $\epsilon = 10^{-12}$ to avoid $\log(0)$, which is undefined.)

What logarithm does:

- $\log_{10}(0.001) = -3$
- $\log_{10}(1) = 0$
- $\log_{10}(100) = 2$

Now our range is from about -3 to +2, which is much easier to work with.

Intuition: Logarithms "compress" large values and "expand" small values. It's like how decibels work for sound—instead of saying "this sound is 1,000,000 times more powerful," we say "it's 60 dB louder."

Visual analogy: Imagine you have a bar chart where one bar is 100,000 units tall and another is 1 unit. You can't see the small bar! Taking logarithm is like using a "zoom" that makes both bars visible.

STEP 4: Baseline Estimation

Now we need to figure out what the "rest" level looks like, so we can set a threshold above it to detect MVCs.

The Challenge

We can't just use the first few seconds as baseline because:

- The participant might not be resting at the start
- There might be noise or artifacts at the beginning
- The recording might start during an MVC

Our Solution: Search for the Quietest Window

We search the **entire recording** to find the quietest half-second.

Algorithm:

1. Slide a 0.5-second window across the signal

2. For each position, compute the **median** of log-energy values in that window
3. The window with the **lowest median** is likely the "rest" period

Why median instead of mean?

- **Mean** is sensitive to outliers—one spike in the window would raise the average
- **Median** is robust—it gives you the "middle" value, ignoring extremes

Analogy: You're trying to find the quietest room in a building. You walk through each room, listen for 30 seconds, and note the "typical" noise level. The median captures the typical level without being thrown off by a single loud cough.

Dropout Detection

Sometimes sensors malfunction and produce a completely flat signal (no variation at all). This would have the lowest median, but it's not valid baseline data!

How we detect this:

- For each candidate baseline window, we check the **IQR (Interquartile Range)** in the original linear energy (before log transform)
- IQR measures the spread of the middle 50% of values
- If IQR is near zero, the signal is suspiciously flat → likely a dropout → skip this window

Why check in linear space? The log transform compresses values, making everything look more similar. Checking IQR in linear space is more sensitive to true dropouts.

STEP 5: Robust Sigma (Noise Level Estimation)

Once we have the baseline window, we need to know how much the "rest" signal naturally fluctuates. This tells us how far above baseline we need to go to confidently say "this is muscle activity, not just noise."

MAD: Median Absolute Deviation

We use **MAD** instead of standard deviation because it's more robust to outliers.

Formula:

$$MAD = \text{median}(|x - \text{median}(x)|)$$

In words: Find the median. Then, for each point, compute how far it is from the median. Take the median of those distances.

Converting to sigma:

$$\sigma_{\text{robust}} = 1.4826 \times MAD$$

The factor 1.4826 makes this equivalent to standard deviation for normally distributed data.

Why not just use standard deviation? Standard deviation uses squared differences, which amplifies outliers. One spike in the baseline could double the standard deviation. MAD is resistant to such outliers.

Adaptive Minimum Guard

Sometimes the baseline is so flat that $MAD \approx 0$. This would cause problems (division by zero, or a threshold that's too close to baseline).

Our safety net:

$$\sigma_{\min} = \max(5\% \times (P90 - P10), \quad 10\% \times IQR_{\text{baseline}}, \quad 0.02)$$

We take the maximum of three values:

1. **5% of dynamic range:** A fraction of the overall signal spread
2. **10% of baseline IQR:** Related to baseline variability
3. **0.02:** An absolute minimum floor in log-space

If the computed MAD-sigma is below this minimum, we fall back to an IQR-based estimate instead.

STEP 6: Computing Candidate Thresholds

Now we have:

- **Baseline median:** The "rest" level of log-energy
- **Robust sigma:** How much natural variation there is at rest

We can now compute thresholds that define "above rest = muscle activity."

Primary Threshold: Baseline + $k \times \sigma$

$$T_{\text{baseline}} = \text{median}_{\text{baseline}} + k \times \sigma_{\text{robust}} \text{ where } (k = 6) \text{ (our default).}$$

This means: "anything more than 6 standard deviations above the baseline median is considered active."

Why 6 sigma? In a normal distribution, 6 sigma catches 99.999998% of the data. So if the noise is normally distributed, there's essentially zero chance of noise exceeding this threshold. However, we're being extra conservative because false positives (calling rest as active) are worse than false negatives.

Secondary Threshold: Otsu's Method

Otsu's method is a classic algorithm from image processing that finds the threshold that best separates two groups.

Intuition: Imagine you have a histogram of log-energy values with two bumps:

- A big bump on the left (rest values)
- A smaller bump on the right (active values)

Otsu finds the valley between these bumps—the optimal dividing point.

How it works (simplified):

1. Try every possible threshold
2. For each threshold, split data into "below" and "above" groups

3. Compute how "tight" each group is (low variance = tight)
4. Pick the threshold that makes both groups as tight as possible

When we use it: Only if Otsu threshold > baseline threshold and falls within a reasonable range. Sometimes Otsu fails (e.g., if the signal is mostly rest with tiny MVCs), so we don't blindly trust it.

Additional Candidates

To handle tricky cases, we also try:

- **Baseline + $(k+2)\sigma$** : Higher threshold if baseline is too sensitive
- **Baseline + $(k+4)\sigma$** : Even higher for very noisy signals
- **75th percentile**: A data-driven threshold
- **85th percentile**: More aggressive, for signals with lots of noise

STEP 7: Building Segments from Thresholds

For each candidate threshold, we build a **binary mask**: 1 where log-energy exceeds threshold, 0 otherwise.

```
binary = (log_energy > threshold) ? 1 : 0
```

This gives us a sequence like: 000000011111111110000001111111000000...

But this raw binary mask has problems:

- Brief threshold crossings create spurious tiny segments
- Brief dips during sustained activity split segments incorrectly

Persistence Filtering

On-persistence (25ms): A segment doesn't officially "start" until the signal has been above threshold for at least 25ms. Brief spikes are ignored.

Off-persistence (25ms): A segment doesn't officially "end" until the signal has been below threshold for at least 25ms. Brief dips during activity don't end the segment.

Analogy: Imagine a security guard who only sounds an alarm if an intruder stays in view for at least 25ms. A bird briefly flying past won't trigger a false alarm.

Gap Merging (150ms)

If two segments are separated by a gap of ≤ 150 ms, we merge them into one.

Why? During an MVC, the muscle might briefly relax slightly, causing a momentary dip below threshold. We don't want to call this "two separate MVCs"—it's really one sustained effort.

Analogy: If someone is talking and pauses for 0.1 seconds to breathe, we don't say they gave two separate speeches.

Minimum Length Filtering (1 second)

Any segment shorter than 1 second is discarded.

Why? An MVC should last at least a second (protocol asks for 2-3 seconds). Brief bursts are likely noise or involuntary twitches, not real MVCs.

STEP 8: Scoring Each Segmentation

This is the clever part. We evaluate how "good" each candidate threshold's segmentation looks.

Base Score: Segment Count

We want exactly **2 segments** (two MVCs). The scoring reflects this:

Segments	Score	Reasoning
2	+25	Perfect—exactly what the protocol expects
3	+15	Good—extra segment but still valid
4+	$10 - 2 \times (n - 4)$	Bad—too many, likely noise contamination
1	-20	Failed—missing one MVC
0	-50	Completely failed—no detection

Key insight: Any threshold that produces ≥ 2 segments will always outscore one that produces < 2 . This aligns with our guardrail that rejects recordings with fewer than 2 segments.

Identifying Top-2 Segments

If we detected more than 2 segments, which two are the "real" MVCs?

We rank segments by **peak energy**—the maximum log-energy value within each segment. The two segments with the highest peaks are our "top-2."

Why peak energy? MVCs are maximum effort—they should have the highest energy peaks. If we detected 5 segments, the true MVCs are probably the two with the biggest peaks; the others might be noise or sub-maximal efforts.

Extra segments beyond the top-2 receive a **-3 penalty each**.

Duration Scoring

MVC contractions should last 1-3 seconds. We score each of the top-2 segments:

Duration	Score	Interpretation
1-3s	+5	Ideal
0.5-1s	+2	Short but acceptable

Duration	Score	Interpretation
3-5s	+2	Long but acceptable
5-10s	-10	Too long—likely merged segments
> 10s	-20 - 2×(d-10)	Way too long—almost certainly wrong

A 30-second segment would get a massive penalty: $-20 - 2 \times 20 = \textbf{-60 points}$.

Contrast Scoring

The top-2 segments should have significantly higher energy than the baseline. We compute:

$$\text{\$contrast} = \text{average}(\text{\text{peak energy of top-2}}) - \text{median}(\text{\text{baseline energy}})\text{\$}$$

Contrast	Score	Interpretation
> 1.5	+5	Excellent—clearly distinct from baseline
> 1.0	+3	Good
< 0.5	-5	Poor—hard to distinguish from noise

Why 1.0 is meaningful: In log space, a difference of 1.0 means the MVC has 10× more energy than baseline (because $10^1 = 10$). That's clearly muscle activity, not noise.

Other Penalties

- **Overlaps baseline window:** -15 (the threshold is so low it detected the baseline as active—clearly wrong)
- **Extends into padded edges:** -5 (edge effects from windowing)

Separation Bonus

If the top-2 segments are spread across >30% of the recording (e.g., one at 10s and one at 50s in a 60s recording), we give +5 bonus.

Why? The protocol asks for two separate MVCs with rest in between. If both segments are clustered together, something might be wrong.

STEP 9: Selecting the Best Threshold

We've now computed scores for all candidate thresholds (baseline, Otsu, baseline_high, p75, p85, etc.).

Simply pick the threshold with the highest score.

This is "evidence-driven" selection: we don't blindly commit to one method. Instead, we let the data tell us which threshold produces the best segmentation.

STEP 10: Final Output

The pipeline returns:

1. **Segments:** List of (start_sample, end_sample) for each detected segment
 2. **Debug info:** All intermediate values for visualization and troubleshooting
-

Part 4: Handling Edge Cases

Case 1: Elevated Noise in Part of Recording

Problem: The first 30 seconds has high baseline noise, but the algorithm finds the quiet baseline later. The threshold computed from the quiet baseline is too low and catches the noisy section as "active."

Solution: Higher threshold candidates (p75, p85, baseline_higher) are evaluated. The scoring heavily penalizes 30-second segments, so these high thresholds win.

Case 2: Sustained Activity

Problem: The participant didn't follow protocol and contracted continuously for 20 seconds instead of doing two separate MVCs.

Solution: The 20-second segment gets a duration penalty. If there are actual MVC peaks within, a higher threshold might separate them. The scoring guides us toward better segmentations.

Case 3: Very Weak MVCs

Problem: The participant didn't contract very hard, so MVCs are barely above baseline.

Solution: The baseline-MAD threshold is specifically designed for low-contrast signals. Otsu might fail, but baseline + 6σ can still detect subtle elevations.

Case 4: Sensor Dropout

Problem: The sensor disconnected briefly, creating a flat-line region that would otherwise appear as the "quietest" baseline.

Solution: Our IQR-based dropout check in linear energy space detects flat regions and skips them when selecting baseline.

Part 5: Why This Design?

Evidence-Driven vs. Fixed Rules

Many algorithms use fixed rules ("threshold = 50% of maximum"). These fail when:

- The signal is noisy
- The MVCs are weak
- The recording quality varies

Our approach evaluates multiple thresholds and picks the one that produces the most MVC-like segmentation. We're not guessing—we're measuring outcomes.

Robustness Stack

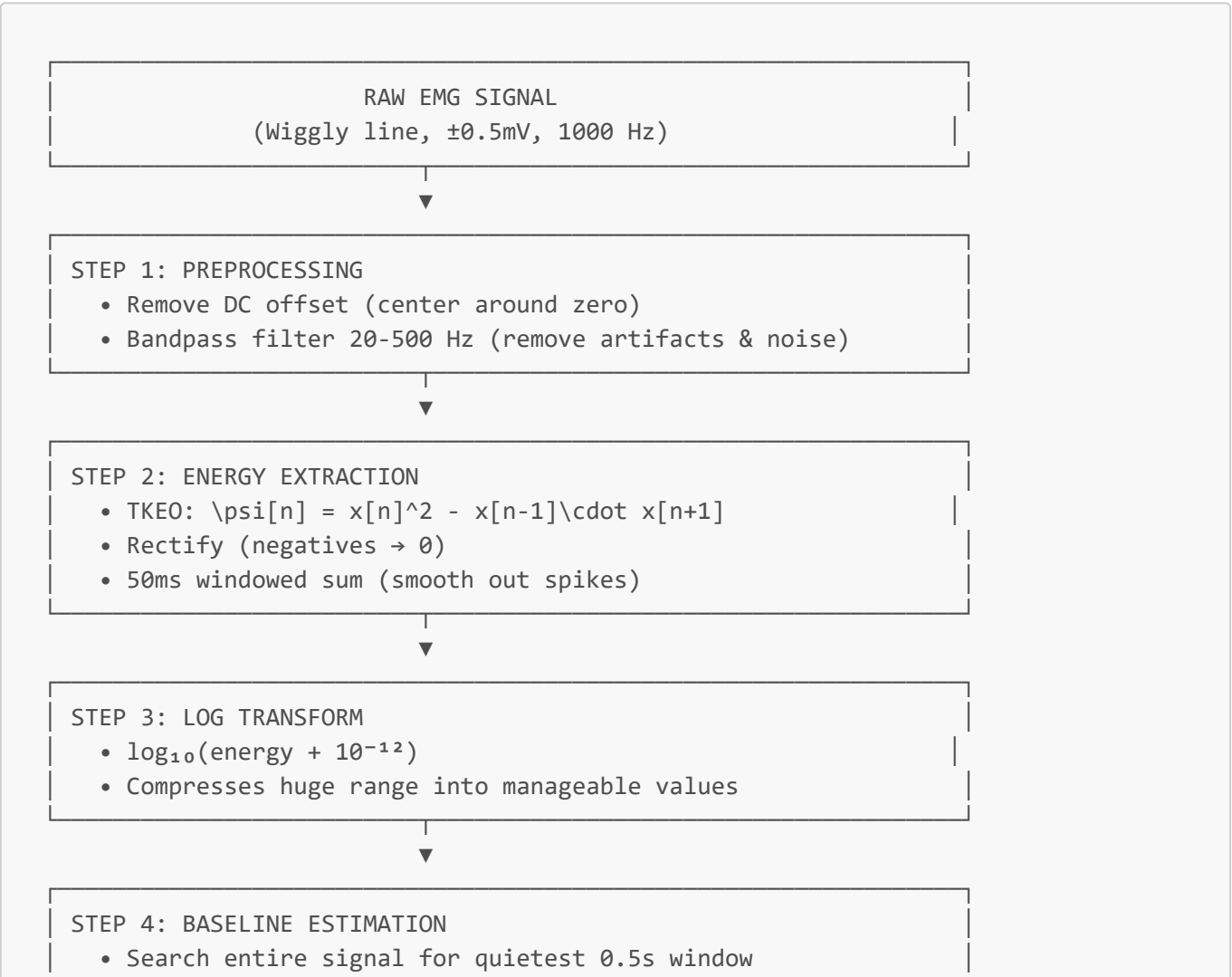
We've built multiple layers of protection:

- 1. **Bandpass filter** → removes motion artifacts and electronic noise
- 2. **TKEO** → enhances sharp transitions, making MVCs more detectable
- 3. **Log transform** → compresses dynamic range, enables consistent thresholding
- 4. **Whole-signal baseline search** → finds true rest regardless of recording start
- 5. **Dropout detection** → ignores sensor failures
- 6. **Adaptive sigma** → handles zero-inflated TKEO baselines
- 7. **Multiple candidate thresholds** → doesn't commit to one potentially wrong method
- 8. **Scoring function** → quantifies what "good" looks like
- 9. **Duration penalties** → rejects obviously wrong segments
- 10. **Top-2 ranking** → identifies true MVCs even with extra segments

Alignment with Downstream Requirements

Our scoring aligns with the pipeline's guardrail: recordings with <2 segments are rejected. By ensuring thresholds that produce 2+ segments always outscore those that produce 1 or 0, we maximize the chance of successful processing.

Part 6: The Complete Flow (Summary)



- Use median to find typical "rest" level
- Dropout check: skip suspiciously flat windows

**STEP 5: ROBUST SIGMA**

- MAD-based noise estimation (resistant to outliers)
- Adaptive minimum guard (prevents $\sigma \approx 0$)

**STEP 6: CANDIDATE THRESHOLDS**

- Baseline: median + 6σ
- Otsu: histogram-based optimal split
- Higher variants: $+8\sigma$, $+10\sigma$
- Percentiles: P75, P85

**STEP 7: BUILD SEGMENTS (for each threshold)**

- Binary mask: $\log_energy > \text{threshold}$
- Persistence filter: 25ms on/off
- Gap merge: $\leq 150\text{ms}$ gaps combined
- Length filter: discard < 1 second segments

**STEP 8: SCORE EACH SEGMENTATION**

- Base score: 2 segments = +25, 3 = +15, 4+ = penalized
- Identify top-2 by peak energy
- Duration: 1-3s = +5, $> 10\text{s}$ = heavy penalty
- Contrast: must be clearly above baseline
- Penalties: baseline overlap, edge effects
- Bonus: well-separated segments

**STEP 9: SELECT BEST THRESHOLD**

- Pick threshold with highest score
- "Evidence-driven" - data tells us what works

**STEP 10: OUTPUT**

- List of segments: $[(\text{start}_1, \text{end}_1), (\text{start}_2, \text{end}_2), \dots]$
- Debug info for visualization

Part 7: Practical Example

Let's trace through a concrete example.

Input: 40-second EMG recording at 1000 Hz (40,000 samples)

- 0-10s: Rest (low noise)
- 10-13s: MVC #1 (high activity)
- 13-25s: Rest
- 25-28s: MVC #2 (high activity)
- 28-40s: Rest

After Step 1 (Preprocessing)

Signal centered around 0, frequencies outside 20-500 Hz removed.

After Step 2 (Energy)

- Rest periods: energy $\approx$ 0.001-0.01
- MVC periods: energy $\approx$ 10-100

After Step 3 (Log Transform)

- Rest periods: log_energy $\approx$ -3 to -2
- MVC periods: log_energy $\approx$ +1 to +2

Step 4 (Baseline)

Quietest window found at $t=5-5.5s$ (during first rest). Median = -2.5.

Step 5 (Sigma)

MAD of baseline = 0.1 $\rightarrow \sigma_{\text{robust}} = 0.148$.

Step 6 (Thresholds)

- Baseline: $-2.5 + 6 \times 0.148 = \mathbf{-1.61}$
- Otsu: **-1.2** (finds valley between rest and active humps)
- P75: **-1.8**
- P85: **-0.9**

Step 7 (Segments)

For baseline threshold (-1.61):

- Segment 1: samples 9,800-13,200 (9.8s-13.2s, duration 3.4s)
- Segment 2: samples 24,700-28,300 (24.7s-28.3s, duration 3.6s)
- $n = 2 \checkmark$

For Otsu threshold (-1.2):

- Same two segments, slightly shorter
- $n = 2 \checkmark$

Step 8 (Scoring)

Baseline threshold score:

- Base: +25 (2 segments)
- Duration Seg1: +2 (3.4s, slightly long)
- Duration Seg2: +2 (3.6s, slightly long)
- Contrast: +5 (peaks at +1.5 vs baseline -2.5 = 4.0 difference, excellent)
- Separation: +5 (centers at 11.5s and 26.5s, well separated)
- **Total: 39**

Otsu threshold score: Similar, maybe slightly different durations → 38.

P85 threshold score:

- Only detects 1 segment (missed second MVC) → base score -20
- **Total: ≈-15**

Step 9 (Selection)

Baseline threshold wins with score 39.

Step 10 (Output)

Segments: [(9800, 13200), (24700, 28300)]

These can be converted to time: MVC #1 from 9.8s to 13.2s, MVC #2 from 24.7s to 28.3s.

Conclusion

This pipeline transforms a noisy, chaotic EMG signal into clean segment boundaries marking the two MVCs. Every step has a purpose:

1. **Clean** the signal (filtering)
2. **Quantify** the activity (TKEO → energy)
3. **Normalize** the scale (log transform)
4. **Establish** the baseline (quiet window search)
5. **Measure** the noise (robust sigma)
6. **Propose** multiple thresholds (candidates)
7. **Apply** thresholds (binary mask → segments)
8. **Evaluate** each result (scoring)
9. **Choose** the best (evidence-driven selection)

The beauty is that we don't rely on any single assumption. If baseline-MAD fails, Otsu might work. If both are too low, percentiles step in. The scoring function acts as an objective referee, picking whichever approach produces the most MVC-like result for that specific recording.